Factorio Planning

ASP-based planning for a simplified *Factorio* model

Anton Althoff (446966)

March 2026

Introduction

- *Factorio* is an automation game centered around building a factory
- Starting from raw resources $\rightarrow$ transform into intermediates $\rightarrow$ final products
- Fundamentally a planning problem: what actions does the player take?
- Focus on temporal scheduling and resource flow
- Abstract away spatial nature $\rightarrow$ simplified model that still preserves most of the core mechanics

Clingcon

- Extends Clingo with **integer variables** and **linear constraints**
- Atoms can be combined with numeric constraints

Example:

```
1 { build_furnace; build_assembler } 1.

&dom{0..10} = iron_used.
&dom{0..10} = copper_used.
&dom{0..20} = total_used.

&sum{ iron_used } = 5 :- build_furnace.
&sum{ iron_used } = 8 :- build_assembler.
&sum{ copper_used } = 2.

&sum{ iron_used; copper_used; -total_used } = 0.
&sum{ total_used } <= 10.
```

Background: Production and crafting

- Resources are transformed according to recipes with inputs, outputs, and a duration
- Recipes can be executed manually by the player or automatically by machines.
- Manual crafting
  - flexible (no infrastructure, low throughput)
  - useful in early stages or for low-volume recipes
- Meaningful progress is only feasible with automation

Background: Automation through buildings

- Automation by constructing buildings → transform items according to recipe
- Once placed, buildings operate in parallel → throughput necessary for large-scale production
- Each building type is a production unit with:
  - a construction **cost**,
  - **input** requirements consumed while operating,
  - **outputs** produced while operating,
  - and a limited **capacity** (building count)
- Machine can only run if its inputs are available and there is enough storage space for its outputs

Example



Figure 1: Building cost



Figure 2: Building input/output

Background: Logistics and bottlenecks

- A part of the difficulty of *Factorio* are the logistics
- Resources must be moved between buildings using belts, inserters, pipes, or similar infrastructure
- $\rightarrow$ spatial constraints, transport delays, and throughput limits.
- Even when the necessary items exist somewhere in the factory, production can stall because the right items do not reach the right machines at the right time

Game screenshot

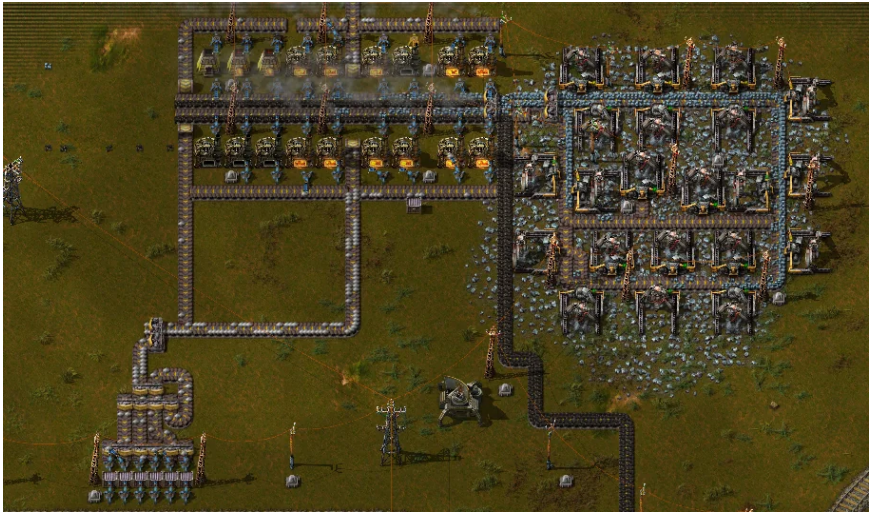


Figure: Screenshot of miner+smelter setup.

Background: Technological progression

- Progression in *Factorio* is gated by research
- Science packs + laboratories $\rightarrow$ new technologies (available recipes, buildings)
- Early-game production $\rightarrow$ advanced outputs
- In this project, technological progression through research is **not** modeled.

Modeling overview

- Model *Factorio* as a discrete-time planning problem using ASP
- Focus on temporal scheduling of building and crafting actions
- Logistic aspects are not modeled explicitly $\rightarrow$ factory regions called **zones**.

Static game encoding

The encoding distinguishes between **crafting recipes** and **buildings** representing automated production units.

```
% items
item(iron_plate).
item(green_circuit).
item(assembler).

% manual crafting recipe
crafting(assembler).
dur(assembler,1).
crafting_cost(assembler, iron_plate, 100).
crafting_cost(assembler, green_circuit, 30).
crafting_out(assembler, assembler, 10).

% automated building
building(green_circuit_assembler).
dur(green_circuit_assembler, 1).
building_cost(green_circuit_assembler, assembler, 5).
building_in(green_circuit_assembler, iron_plate, 100).
building_in(green_circuit_assembler, electricity, 78).
building_out(green_circuit_assembler, green_circuit, 100).
```

Simplifications

The model uses fixed-length time horizons (e.g., 20-30 steps), each step corresponds to 30 seconds of in-game time

- Spatial placement is abstracted away and replaced by zones.
- Logistic infrastructure is not modeled explicitly.
- All recipes are available from the start
- Recipes are treated at aggregate rates per step
- At most one manual crafting action and one building action per step

Actions and delayed completion

At each step, the player may start up to one crafting action and one building action. Once started, the respective activity channel remains occupied until the duration expires.

```
% at most one craft and one build action per step
0 { act(craft(C), STEP) : crafting(C), dur(C, _) } 1
  :- step(STEP), not busy(craft, STEP).

0 { act(build(B, Z), STEP) : building(B), zone(Z),
    dur(B, _), can_build(B, Z) } 1
  :- step(STEP), not busy(build, STEP).

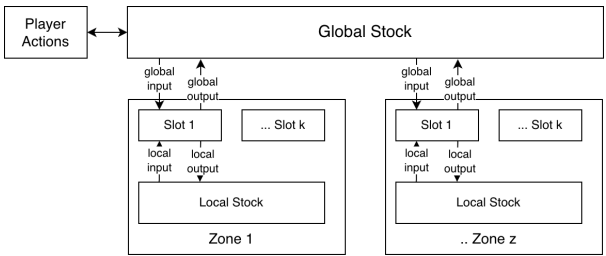
% activity channels stay occupied until the duration expires
busy(craft, STEP) :- step(STEP), act(craft(C), STEP0),
  dur(C, D), STEP > STEP0, STEP < STEP0 + D.

% completion time
finish(craft(C), TIME) :- time(TIME), act(craft(C), STEP0),
  dur(C, D), TIME = STEP0 + D.
```

Craft and build channels are handled symmetrically in the full encoding.

Zones

- The factory is partitioned into **zones**.
- Each zone contains a fixed number of **slots**.
- A slot may be assigned at most one building type.
- A building type may appear in at most one slot per zone.
- The slot assignment is fixed over the horizon.
- `building_count(B,Z,T)` represents the installed capacity.



Zone resource flow model.

Zones

```
zone(1..num_zones).
slot(1..num_slots).
available_index(1..num_slots+1).
next_zone(Z, Z+1) :- zone(Z), zone(Z+1).

% at most one building type per slot
0 { slot_map(Z, S, B) : building(B) } 1 :- zone(Z), slot(S).

% a building type may occupy at most one slot per zone
:- slot_map(Z, S1, B), slot_map(Z, S2, B), S1 != S2.

% keep used slots left-aligned
has_assignment(Z, S) :- slot_map(Z, S, _).
:- has_assignment(Z, S+1), not has_assignment(Z, S), slot(S).

% build actions only for assigned building types
can_build(B, Z) :- building(B), zone(Z), slot_map(Z, _, B).
```

Time model and state variables

- The factory is modeled over a fixed horizon with `time(0..H)` and `step(0..H-1)`.
- A **time point** represents a snapshot of the state.
- A **step** represents the transition between two consecutive snapshots.
- One step corresponds to 30 seconds of in-game time.

At each time point T, the relevant part of the state is:

- `global_stock(I,T)`: global stock of item I,
- `stock(I,Z,T)`: local stock of item I in zone Z,
- `building_count(B,Z,T)`: number of buildings of type B in zone Z.

Intra-step resource allocation: initialization

Within one step, multiple buildings may compete for the same limited resources. To remove ambiguity, the model fixes the order in which zones and slots gain access to available resources.

```
uses_build_resource(I, T) :- act(build(B, _), T), building_cost(B, I, _).
uses_craft_resource(I, T) :- act(craft(C), T), crafting_cost(C, I, _).

% each zone starts with its local stock available to slot 1
&sum { local_available(I, Z, 1, T); -stock(I, Z, T) } = 0
  :- item(I), zone(Z), step(T).

% first slot of first zone receives remaining global stock
&sum { global_available(I, 1, 1, T); -global_stock(I, T);
       cost_build(I, T); cost_craft(I, T) } = 0
  :- item(I), step(T).

% available output space
&sum { local_space(I, Z, 1, T); stock(I, Z, T) } = Cap
  :- item(I), zone(Z), step(T), init_local_cap(I, Z, Cap).
&sum { global_space(I, 1, 1, T); global_stock(I, T) } = Cap
  :- item(I), step(T), init_global_cap(I, Cap).
```

Core intra-step flow constraints

For each occupied slot, the solver chooses a utilization value between zero and installed capacity. This utilization is constrained by available inputs and available output space.

```
&dom{0..max_val} = slot_utilization(Z, S, T) :- zone(Z), slot(S), step(T).

% utilization cannot exceed installed capacity
&sum { slot_utilization(Z, S, T); -building_count(B, Z, T) } <= 0
  :- slot_map(Z, S, B), step(T).

% required inputs = recipe demand * utilization
&sum { input_from_local(I, Z, S, T); input_from_global(I, Z, S, T);
       -Qty*slot_utilization(Z, S, T) } = 0
  :- slot_map(Z, S, B), building_in(B, I, Qty), step(T).

% produced outputs = recipe output * utilization
&sum { output_to_local(I, Z, S, T); output_to_global(I, Z, S, T);
       -Qty*slot_utilization(Z, S, T) } = 0
  :- slot_map(Z, S, B), building_out(B, I, Qty), step(T).
```

Availability and storage-space constraints bound these flows in the full encoding.

Propagation and state transition

After a slot's utilization is fixed, the remaining local resources, global resources, and output space are passed forward. At the end of the step, the model fixes the state at time $T + 1$.

```
% pass remaining resources to next slot
&sum { local_available(I, Z, S+1, T); -local_available(I, Z, S, T);
       input_from_local(I, Z, S, T) } = 0
  :- item(I), zone(Z), slot(S), step(T).

% building capacity increases when build action finishes
&sum { building_count(B, Z, T+1); -building_count(B, Z, T) } = 1
  :- building(B), zone(Z), step(T), slot_map(Z, _, B), finish(build(B, Z), T).

% local stock at T+1 = remaining local availability + outputs to local stock
&sum { stock(I, Z, T+1); -local_available(I, Z, num_slots+1, T);
       -output_to_local(I, Z, S, T) : slot(S) } = 0
  :- item(I), zone(Z), step(T).

% global stock at T+1 = remaining global availability + outputs to global + craft yield
&sum { global_stock(I, T+1); -global_available(I, num_zones, num_slots+1, T);
       -output_to_global(I, Z, S, T) : zone(Z), slot(S);
       -yield_craft(I, T+1) } = 0 :- item(I), step(T).
```

Optimization objective

The model does not force greedy utilization per slot. Instead, utilization is selected by the solver subject to the constraints and the optimization objective.

```
% want factories to run as much as possible, consume local resources first
&maximize {
 100*slot_utilization(Z, S, T) : zone(Z), slot(S), step(T);
 -1*input_from_global(I, Z, S, T) : item(I), zone(Z), slot(S), step(T)
}.
```

- First priority: keep factories running as much as possible.
- Second priority: prefer local resource usage over the global pool.

Planning from existing game states

An instance specifies the starting global stock, local zone stocks, installed buildings, and one or more target conditions.

```
#const horizon = 15.
#const num_zones = 2.
#const num_slots = 8.

init_global_stock(electricity, 500).
init_global_stock(coal, 500).
init_building_count(offshore_pump, 1, 1, 1).
init_building_count(power_plant_small, 1, 2, 2).
init_building_count(coal_mine, 1, 3, 1).
init_building_count(iron_mine, 1, 4, 1).

goal_item(green_circuit, 100).
goal_building(green_circuit_assembler, 1).
```

- Run with **clingcon**.
- Output is either **UNSAT** or a sequence of actions achieving the goals within the horizon.
- green_circuit.lp
  - $H = 14$: **UNSAT**
  - $H = 15$: viable solution

```
$ clingcon --stats model.lp game.lp \
  instances/{name} --opt-mode=opt \
  -n 0 --quiet=1 -t 64 --time 1200
```

Output

Solver output (excerpt):

```
Answer: 1 (Time: 227.251s)

act(craft(assembler),10)
act(craft(copper_cable),5) act(craft(copper_cable),6)
...
act(build(iron_smelter,2),3) act(build(iron_smelter,2),4)
act(build(steel_smelter,2),10)
...
finish(craft(stone_furnace),4) finish(craft(stone_furnace),5)
finish(craft(stone_furnace),14)
...
finish(build(copper_cable_assembler,2),12)
finish(build(steel_smelter,2),11)
...
slot_map(1,1,offshore_pump) slot_map(1,2,power_plant_small)
slot_map(2,8,battery_chemical_plant)
```

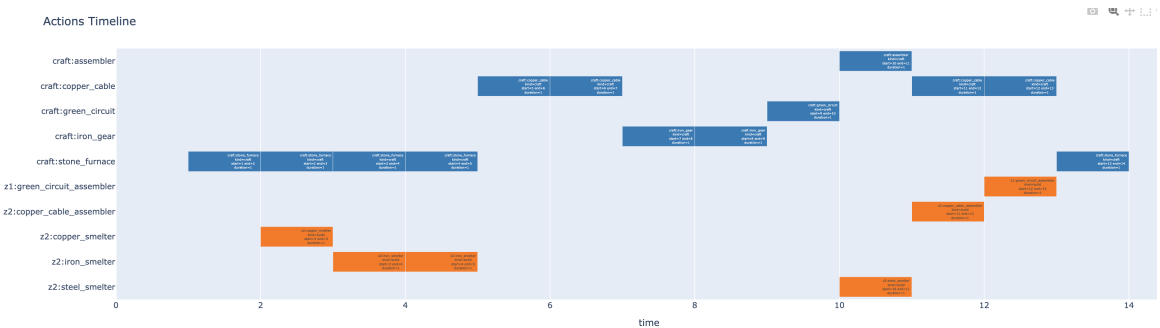


Figure: Action timeline

Conclusion

- Planning model for a simplified version of *Factorio*
- Test whether a goal is reachable within a given horizon
- Find action sequences that achieve the specified objectives

Questions?